ZSUT

Zakład Sieci i Usług Teleinformatycznych

Kubernetes laboratory

Lab 3: Monitoring Open5GS and scaling the UPF function

Prepared by: dr inż. Dariusz Bursztynowski

ZSUT. Zakład Sieci i Usług Teleinformatycznych
Instytut Telekomunikacji
Wydział Elektroniki i Technik Informacyjnych
Politechnika Warszawska

Last update: April 2026

WARNING: Before powering up your devices, make sure you are using the correct power adapter. The power adapters in the boxes have the same DC plug type, but have **COMPLETETLY DIFFERET output voltages** (WiFi routers like Linksys are powered by 12V DC, while the TP-Link switch requires 53V DC). Powering your WiFi router from the TP-Link switch power adapter will **immediately damage the router**.

Table of contents

1. **The goal** 3
2. **Service metric monitoring** 3
3. **The report** 3
4. **Useful readings and videos** 4

1. The goal

This lab aims to provide insight into the differences between resource and service level monitoring in the context of resource and service orchestration (we remind that resource monitoring was included in Lab 2 P3). For obvious reasons, the demonstration itself is very simple. Nevertheless, we hope it will be enough to gain the basic understanding of the nature of management at both levels. The bonus is that you deploy and manage in your Kubernetes cluster a 5G network instance consisting of a 5G backbone network (based on the Open5GS platform) and a popular radio network emulator UERANSIM.

2. Service metric monitoring

Service metric monitoring experiment and all relevant details related to our 5G network platform are described **in this repo** which is an integral part of the lab. Refer there to complete it. Important: use the branch ***upf-scaling-spiw***.

The main goal of this lab is to demonstrate how telecom platform resources can be orchestrated (managed) based on service level metrics. As already mentioned, this should be clearly distinguished from resource-level management/orchestration based on resource-level metrics (as CPU or RAM usage).

The scenario involves monitoring partially emulated 5G network. The RAN part of the network will be emulated using UERANSIM, while the 5G core will be built upon full-fledged containerized Open5GS platform. The deployment of both components will be based on slightly customized Gradiant Helm charts.

During the lab, we will briefly discuss the configuration and implementation of a 5G network to understand the structure of our implementation against the background of the 5G network architecture. However, we will only cover the absolute minimum necessary to run the 5G environment and manage the number of active terminals in the network. Then we will monitor a selected service-level metric using Prometheus. This will underpin our understanding of service monitoring in preparation to **Project 2**.

> *In the project, we will design and deploy a simple resource level orchestrator operating based on service-level metrics. It will monitor the number of user sessions in the network and scale vertically the UPF[1] (User Plane Function) container. The scaling operation will be triggered when the number of user sessions exceeds the predefined thresholds. Since our primary focus is on service metric monitoring and resource scaling, we won't delve too deeply into the details of how the 5G core works. However, students with a deeper interest in orchestration will also be able to experiment with their own scaling algorithms and with observability tools, such as designing their own Grafana dashboards.*

3. The report

At a minimum, the report should include proof of project completion in the form of screenshots of the manually scaled UPF, and a summary of key findings and lessons learned.

[1] In 5G core network architecture, UPF is the function responsible for terminating GTP tunnels that carry user traffic to/from user terminals. It handles all data plane traffic (in volume, dominant traffic in the network). In the basic scenario, those tunnels span between UPF nodes and gNB nodes (the base stations).

4. Useful readings and videos

(Note: new valuable stuff is appearing all the time so one can find other interesting sources.)

Kubernetes troubleshooting
https://lumigo.io/kubernetes-troubleshooting/
https://www.groundcover.com/kubernetes-troubleshooting
https://komodor.com/learn/kubernetes-troubleshooting-the-complete-guide/

- kubectl cheat-sheet
https://kubernetes.io/docs/reference/kubectl/cheatsheet/

- delete all resources from namespace
https://stackoverflow.com/questions/47128586/how-to-delete-all-resources-from-kubernetes-one-time

- How to Monitor a Kubernetes Cluster in 2022 with Prometheus & Grafana
https://www.youtube.com/watch?v=YDtuwlNTzRc

- Modern Grafana dashboards for kubernetes
https://medium.com/@dotdc/a-set-of-modern-grafana-dashboards-for-kubernetes-4b989c72a4b2

- Adding Grafana dashboard using configMap
https://fabianlee.org/2022/07/06/prometheus-adding-a-grafana-dashboard-using-a-configmap/

- Simple Prometheus & Grafana setup with PVC
https://medium.com/@bmbvfx/kubernetes-persistent-volume-claim-prometheus-grafana-4e821e283edc

- Accessing Kubernetes services
OK: https://blog.alexellis.io/primer-accessing-kubernetes-services/
https://kubernetes.io/docs/tasks/access-application-cluster/access-cluster-services/

Prometheus/AlertManager/Grafana/Loki - ciekawe, do pooglądania, wykorzystanie AlertManager
https://www.youtube.com/watch?v=NABZqKq1McE
przy okazji: strona z opisami helm-chartów dla aplikacji - B. CIEKAWA
https://artifacthub.io/

CNCF landscape
https://landscape.cncf.io/

Kubernetes API Basics - Resources, Kinds, and Objects
https://iximiuz.com/en/posts/kubernetes-api-structure-and-terminology/